PROJET

Worldwide Weather Watcher

Station météo embarquée sur STM32 NUCLEO-L476RG

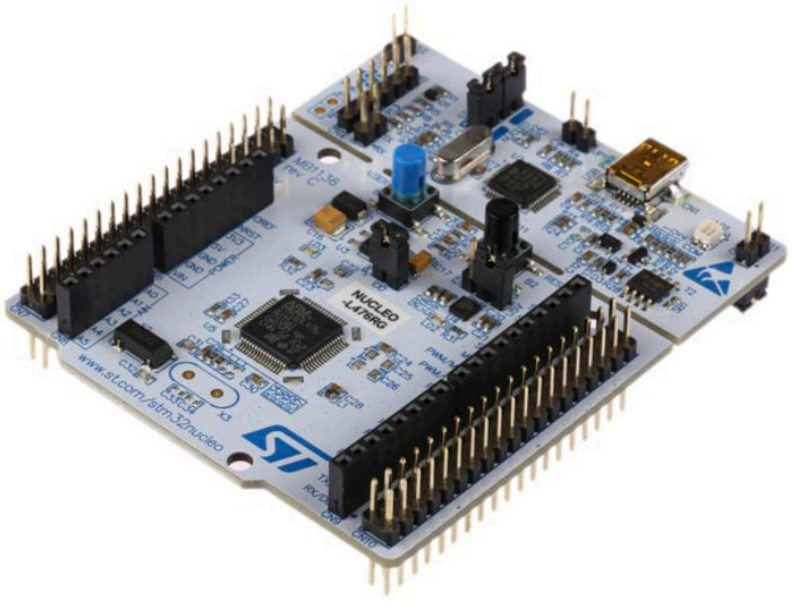
Livrable 02

Architecture du programme : structure générale en algorithmique

Groupe 01

LEBBOUZ Riad · KOUIDER ETTOUMI Mohamed El Amine

HAMCHAOUI Razane · BELOUCHRANI Mustapha



STM32 NUCLEO-L476RG

Carte : STM32 NUCLEO-L476RG (Cortex-M4, 80 MHz) · **Langage :** C (HAL STM32 + FatFs)

Capteurs : DHT11, BH1750, GPS, RTC interne · **Stockage :** carte micro-SD

Interface : LED RGB d'état, 2 boutons poussoirs, console série 115200 bauds

Octobre 2026

Sommaire

1 Présentation du projet et objectif du livrable

2 Matériel utilisé

3 Architecture générale du programme (modules et fichiers)

4 Constantes, types et variables globales

5 Les 4 modes de fonctionnement (diagramme d'états)

6 Algorithme principal

7 Algorithmes des modes et des boutons

8 Algorithmes d'acquisition des capteurs

9 Enregistrement sur carte SD

10 Configuration et console série

11 LED d'état et gestion des erreurs

12 Récapitulatif des fonctions par module

13 Conclusion et suite du projet

1. Présentation du projet

Le projet **Worldwide Weather Watcher** consiste à réaliser une station météo autonome. La carte STM32 NUCLEO-L476RG mesure à intervalle régulier la **température** et l'**humidité** de l'air (DHT11), la **luminosité** (BH1750) et la **position** (module GPS). Chaque mesure est horodatée grâce à l'horloge temps réel (RTC) interne du microcontrôleur, puis enregistrée dans un fichier sur une **carte micro-SD**.

L'utilisateur interagit avec la station grâce à **deux boutons** qui font passer d'un mode à l'autre, une **LED RGB** qui indique le mode courant ou une erreur, et une **console série** qui permet de modifier les paramètres en mode configuration.

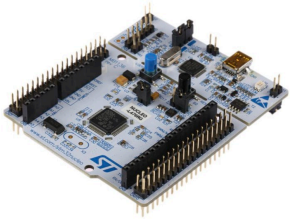
> **Objectif du livrable 02.** Présenter l'architecture du programme de la carte : découpage en modules, fonctions, variables et structures de données. Les algorithmes sont volontairement écrits en **pseudo-code simple et non optimisé** : ils décrivent la logique attendue et non un programme finalisé. L'optimisation (consommation, mise en veille, lecture non bloquante des capteurs) fera l'objet d'une étape ultérieure.

Conventions d'écriture du pseudo-code

Notation	Signification
`FONCTION Nom(param) : type`	Sous-programme qui renvoie une valeur
`PROCEDURE Nom(param)`	Sous-programme sans valeur de retour
`SI … ALORS … SINON … FIN SI`	Test conditionnel
`TANT QUE … FAIRE … FIN TANT QUE`	Boucle conditionnelle
`POUR i DE a À b FAIRE … FIN POUR`	Boucle avec compteur
`SELON x … CAS … FIN SELON`	Choix multiple
`←`	Affectation
`// texte`	Commentaire

2. Matériel utilisé

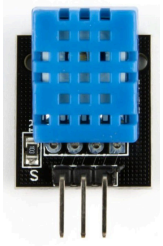
Le tableau des broches correspond au fichier CubeMX `StationMeteo.ioc` du projet. Chaque composant a été testé individuellement (programmes `test_*.c`) avant l'intégration.



STM32 NUCLEO-L476RG

Carte de développement à microcontrôleur ARM Cortex-M4, cadencée à 80 MHz (HSI 16 MHz + PLL). Elle exécute le programme, contient la RTC interne (quartz LSE 32,768 kHz), la mémoire flash qui sert à sauvegarder les paramètres, et le bouton B1 (bouton « rouge » du projet).

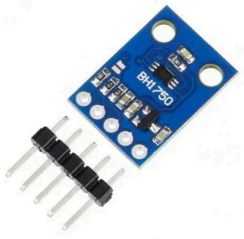
Cœur du système · USART2 → console 115200 bauds · B1 = PC13



DHT11 : température et humidité

Capteur numérique sur un seul fil de données. Le microcontrôleur envoie un signal de départ puis lit 40 bits (humidité, température, somme de contrôle). Plage : 0 à 50 °C, 20 à 90 % HR.

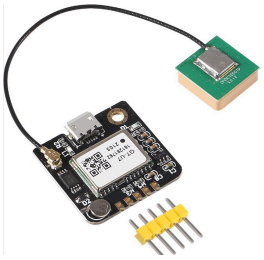
Bus 1 fil · PB0 (A3) · ≥ 2 s entre deux lectures



BH1750 : luminosité

Capteur de lumière ambiante sur bus I2C, adresse 0x23, en mode continu haute résolution (1 lux). Valeur en lux = valeur brute / 1,2.

I2C1 100 kHz · SCL = PB8 (D15) · SDA = PB9 (D14)



Module GPS

Envoie en continu des trames texte NMEA (`$GPGGA`, `$GPRMC`…) sur une liaison série. Le programme reçoit les caractères par interruption et extrait la latitude et la longitude.

USART3 9600 bauds 8N1 · TX = PC10 · RX = PC11



Lecteur de carte micro-SD

Stockage des mesures dans des fichiers `AAMMJJ_n.LOG` via la bibliothèque FatFs (FAT32). La carte est démontée en mode maintenance pour pouvoir être retirée.

SPI1 · SCK PA5 · MISO PA6 · MOSI PA7 · CS = PB6 (D10)



LED RGB (anode commune)

Indique le mode courant par sa couleur et signale les erreurs par un clignotement à 1 Hz. Pilotée en PWM logiciel (10 niveaux d'intensité) depuis l'interruption SysTick toutes les 1 ms.

R = PA0 (A0) · V = PA1 (A1) · B = PA4 (A2) · Allumée = broche à 0



Bouton poussoir miniature

Bouton « vert » du projet (le bouton « rouge » est B1 sur la carte). Les deux boutons sont câblés en pull-up interne : appuyé = niveau 0. Un appui long de 5 s change de mode.

PC1 (A4) · Entrée pull-up · Anti-rebond 30 ms

3. Architecture générale du programme

Le programme est découpé en **modules** : chaque module correspond à un fichier `.c` et à son en-tête `.h`. Le programme principal ne fait qu'initialiser la carte puis appeler en boucle le gestionnaire de modes, qui utilise les autres modules.

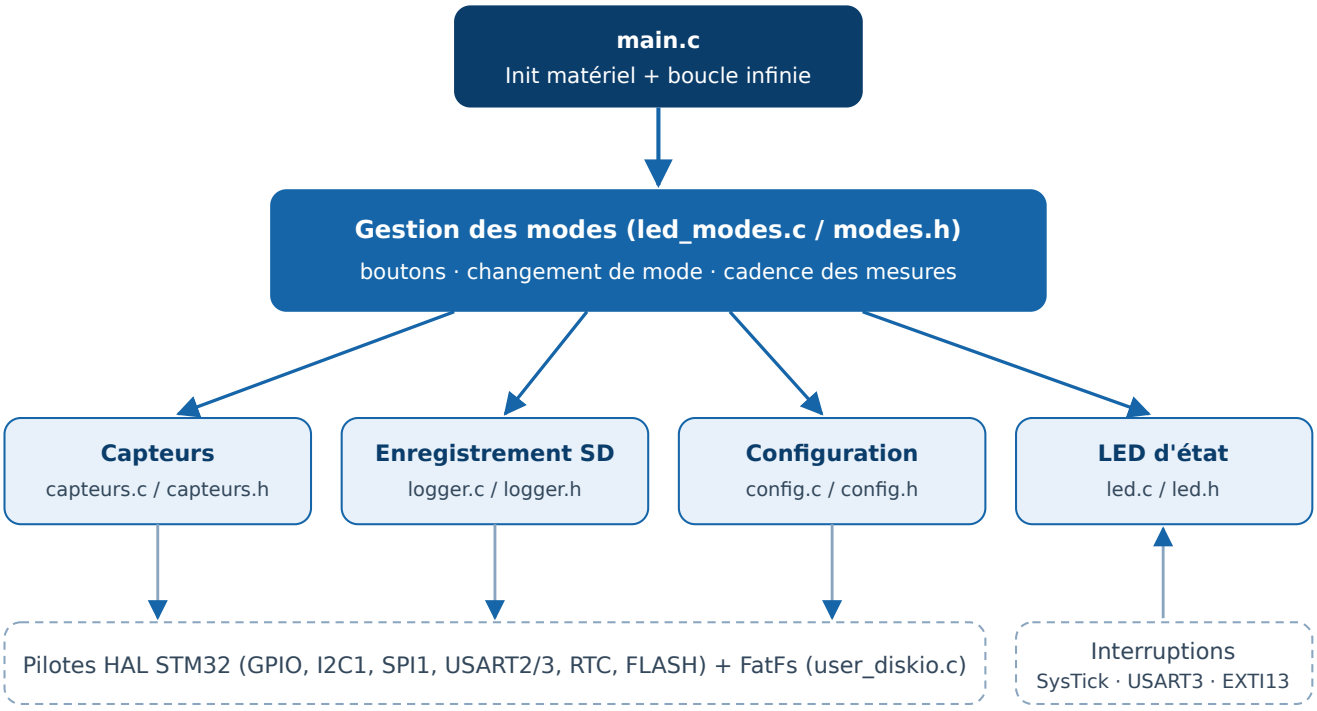


Figure 1 : découpage du programme en modules et dépendances

3.1 Arborescence des fichiers (Core/)

Fichier	Rôle
`main.c / main.h`	Point d'entrée : initialisation des horloges et des périphériques (générée par CubeMX), choix test/station, boucle infinie
`led_modes.c / modes.h`	Machine à états des 4 modes, lecture des boutons, cadence des mesures, console
`capteurs.c / capteurs.h`	Acquisition non bloquante DHT11, BH1750, GPS, contrôle des seuils, mise en forme de la ligne de mesures
`logger.c / logger.h`	Montage de la carte SD, écriture des lignes, archivage des fichiers pleins
`config.c / config.h`	Paramètres de la station, sauvegarde en flash, commandes de la console (CLOCK, DATE, RESET…)
`led.c / led.h`	Couleur de la LED RGB et codes d'erreur clignotants (PWM logiciel)
`stm32l4xx_it.c`	Routines d'interruption : SysTick (appelle `LED_Tick`), USART3 (appelle `Capteurs_GPS_IRQ`), EXTI13 (B1)
`test_*.c / test_composants.h`	Programmes de test unitaires de chaque composant (LED, boutons, série, DHT11, RTC, BH1750, SD, GPS)
`FATFS/Target/user_diskio.c`	Pilote bas niveau de la carte SD en SPI pour FatFs

3.2 Principe de fonctionnement

- Le programme tourne dans une **boucle infinie** (« super-boucle ») sans système d'exploitation.
- Le temps est mesuré avec `HAL_GetTick()` (millisecondes depuis le démarrage). On compare l'heure actuelle à l'heure de la dernière action pour savoir s'il faut agir (boutons, mesures, délai de la configuration).
- Deux tâches sont faites **sous interruption** : le rafraîchissement de la LED (toutes les 1 ms) et la réception des caractères du GPS.
- La date et l'heure des mesures viennent de la **RTC**, qui continue de compter même après un reset.

4. Constantes, types et variables globales

4.1 Constantes

Constante	Valeur	Rôle
`VERSION_PROGRAMME`	"WWW v1.0"	Affichée au démarrage et par la commande VERSION
`NUMERO_LOT`	"LOT-001"	Numéro de lot de la station
`APPUI_LONG_MS`	5000	Durée d'un appui long pour changer de mode
`ANTI_REBOND_MS`	30	Temps de stabilisation d'un bouton
`CONFIG_TIMEOUT_MS`	30 min	Retour automatique en mode standard sans activité
`MAINTENANCE_PERIODE_MS`	5000	Période d'affichage des mesures en maintenance
`BH1750_ADRESSE`	0x23	Adresse I2C du capteur de luminosité
`GPS_BUFFER_SIZE`	128	Taille maximale d'une trame NMEA
`CFG_ADRESSE`	0x080FF800	Dernière page de flash (2 Ko) où sont stockés les paramètres
`CAPTEUR_PRESSION_PRESENT`	0	Pas de capteur de pression câblé : valeur notée « NA »

4.2 Paramètres modifiables (structure `Config`)

Ces paramètres sont modifiables en mode configuration et conservés en flash après une coupure d'alimentation.

Paramètre	Défaut	Min … Max	Rôle
`LOG_INTERVAL`	10	1 … 1440	Minutes entre deux mesures
`FILE_MAX_SIZE`	2048	256 … 1 048 576	Taille max d'un fichier LOG (octets)
`TIMEOUT`	30	1 … 300	Délai max d'acquisition d'un capteur (s)
`LUMIN` / `LUMIN_LOW` / `LUMIN_HIGH`	1 / 255 / 768	0…1 / 0… 1023	Activation et seuils de luminosité
`TEMP_AIR` / `MIN_TEMP_AIR` / `MAX_TEMP_AIR`	1 / −10 / 60	0…1 / −40… 85	Activation et seuils de température (°C)
`HYGR` / `HYGR_MINT` / `HYGR_MAXT`	1 / 0 / 50	0…1 / −40… 85	Activation et plage de température où l'hygrométrie est valide
`PRESSURE` / `PRESSURE_MIN` / `PRESSURE_MAX`	1 / 850 / 1080	0…1 / 300… 1100	Activation et seuils de pression (hPa)

4.3 Types (énumérations et structures)

```
TYPE ModeSysteme = ENUM { MODE_STANDARD, MODE_CONFIGURATION, MODE_ECONOMIQUE,
MODE_MAINTENANCE }

TYPE LedErreur = ENUM { AUCUNE, ERR_RTC, ERR_GPS, ERR_CAPTEUR, ERR_INCOHERENT,
ERR_SD_PLEINE, ERR_SD }

TYPE SdEtat = ENUM { SD_OK, SD_ERREUR, SD_PLEINE }

TYPE Mesures = STRUCTURE
    temp_ok : booléen        temp     : entier   // °C
    hygr_ok : booléen        hygr     : entier   // %
    pres_ok : booléen        pression : entier   // hPa (toujours NA)
    lum_ok  : booléen        lux      : entier   // lux
    gps_lu  : booléen                            // GPS demandé pour cette mesure
    gps_ok  : booléen        gps      : chaîne   // "4851.1234N,00221.5678E"
FIN STRUCTURE

TYPE CapteursErreurs = STRUCTURE
    capteur    : booléen     // un capteur a fait 2 timeouts de suite
    gps        : booléen     // le GPS a fait 2 timeouts de suite
    incoherent : booléen     // une valeur est hors des seuils MIN/MAX
FIN STRUCTURE

TYPE Bouton = STRUCTURE
    port, broche : GPIO
    brut         : booléen   // dernière lecture
    enfonce      : booléen   // état après anti-rebond
    long_fait    : booléen   // appui long déjà signalé
    t_change     : entier    // instant du dernier changement (ms)
    t_appui      : entier    // instant du début de l'appui (ms)
FIN STRUCTURE
```

4.4 Variables globales

Variable	Type	Module	Rôle
`cfg`	Config	config.c	Paramètres courants de la station
`mode`	ModeSysteme	led_modes.c	Mode courant
`mode_precedent`	ModeSysteme	led_modes.c	Mode à reprendre en sortant de maintenance
`rouge, vert`	Bouton	led_modes.c	État des boutons B1 et miniature
`t_mesure`	entier (ms)	led_modes.c	Instant de la dernière mesure
`t_activite`	entier (ms)	led_modes.c	Dernière activité en mode configuration
`mesure_immediate`	booléen	led_modes.c	Mesure dès l'entrée dans un mode
`num_mesure`	entier	led_modes.c	Compteur de mesures (GPS une fois sur deux en économique)
`ligne, lg`	chaîne, entier	led_modes.c	Commande en cours de saisie dans la console
`mesures`	Mesures	capteurs.c	Résultat de la dernière acquisition
`erreurs`	CapteursErreurs	capteurs.c	Erreurs détectées sur les capteurs
`nb_timeout[]`	tableau d'entiers	capteurs.c	Timeouts consécutifs par capteur

gps_rx_buffer, gps_trame, gps_index, gps_data_ready	chaînes, entier, booléen	capteurs.c	Trame GPS reçue sous interruption
en_cours, t_debut, dht_fini, lum_fini, gps_fini	booléens, entier	capteurs.c	Suivi de l'acquisition en cours
t_horodatage, d_horodatage	RTC_Heure, RTC_Date	led_modes.c	Heure du début de la mesure en cours
etat_sd, erreur_rtc	SdEtat, booléen	led_modes.c	Dernier résultat SD et RTC (pour la LED)
fs, monte	FATFS, booléen	logger.c	Système de fichiers de la carte SD et état monté
niv_r, niv_g, niv_b	entiers 0..10	led.c	Intensité de chaque couleur de la LED
erreur, actif, ms	LedErreur, booléen, entier	led.c	Code d'erreur affiché, LED activée, compteur ms
hi2c1, hspi1, huart2, huart3, hrtc	handles HAL	main.c	Périphériques I2C, SPI, séries et RTC

5. Les 4 modes de fonctionnement

Mode	LED	Acquisition	Enregistrement	Particularités
Standard	Verte	Toutes les `LOG_INTERVAL` min, GPS à chaque mesure	Carte SD	Mode par défaut au démarrage
Configuration	Jaune	Désactivée	Aucun	Console série : modification des paramètres, de l'heure et de la date. Retour en standard après 30 min sans activité
Économique	Bleue	Toutes les 2 × `LOG_INTERVAL` min, GPS une mesure sur deux	Carte SD	Réduit la consommation
Maintenance	Orange	Toutes les 5 s	Port série uniquement	Carte SD démontée : retrait possible sans corrompre les fichiers

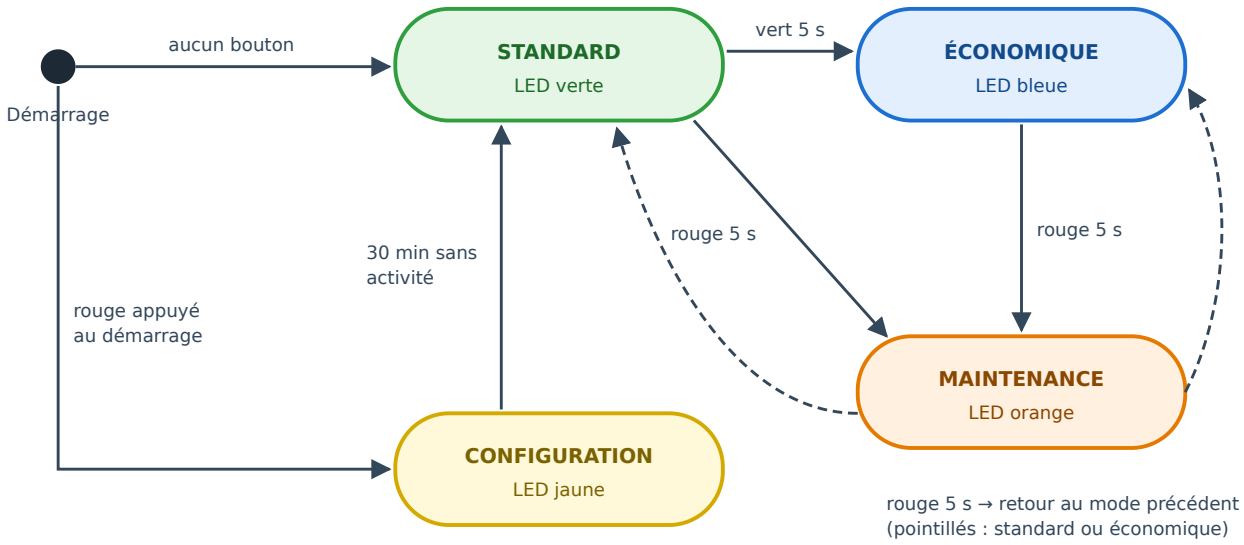


Figure 2 : diagramme d'états. Bouton « rouge » = B1 (PC13), bouton « vert » = bouton miniature (PC1)

> Un appui n'est pris en compte comme **appui long** qu'une seule fois, au moment où il atteint 5 s. L'appui fait au démarrage (pour entrer en configuration) n'est pas compté comme appui long.

6. Algorithme principal

```
ALGORITHME StationMeteo_WWW

DEBUT
    // 1. Initialisation du microcontrôleur
    HAL_Init()                          // compteur SysTick 1 ms
    Configurer_Horloge()                // HSI 16 MHz -> PLL -> 80 MHz, LSE 32,768 kHz pour
la RTC
    Init_GPIO()                         // LED R/V/B en sortie, boutons en entrée pull-up, CS
SD, DHT11
    Init_I2C1()                         // BH1750
    Init_RTC()                          // ne remet pas l'heure à zéro si elle a déjà été
réglée
    Init_SPI1()                         // carte SD
    Init_USART2()                       // console série 115200 bauds
    Init_USART3()                       // GPS 9600 bauds
    Init_FATFS()

    // 2. Choix : programme de test d'un composant OU station complète
    SI TEST_MODE ≠ AUCUN ALORS
        Lancer_Test(TEST_MODE)          // Test_LED, Test_DHT11, Test_GPS ... (ne revient
jamais)
    FIN SI

    // 3. Station météo
    Modes_Init()

    TANT QUE VRAI FAIRE
        Modes_Boucle()
    FIN TANT QUE
FIN
```

```
PROCEDURE Modes_Init()
DEBUT
    Config_Charger()                        // paramètres depuis la flash (ou valeurs par
défaut)
    Capteurs_Init()
    LED_Activer()
    Attendre(50 ms)                         // stabilisation des pull-up

    rouge.enfonce ← Lire_Bouton(rouge)
    vert.enfonce  ← Lire_Bouton(vert)
    rouge.long_fait ← VRAI                  // l'appui du démarrage n'est pas un appui long
    vert.long_fait  ← VRAI

    Afficher("Station meteo ", VERSION_PROGRAMME)

    SI rouge.enfonce ALORS
        Changer_Mode(MODE_CONFIGURATION)
    SINON
        Logger_Monter()
        Changer_Mode(MODE_STANDARD)
    FIN SI
FIN
```

```
PROCEDURE Modes_Boucle()
VARIABLES maintenant : entier ; long_rouge, long_vert : booléen
DEBUT
    maintenant ← HAL_GetTick()
    long_rouge ← Appui_Long(rouge, maintenant)
    long_vert  ← Appui_Long(vert, maintenant)

    SELON mode
        CAS MODE_CONFIGURATION :
            Console(maintenant)
            SI rouge.enfonce OU vert.enfonce ALORS t_activite ← maintenant FIN SI
            SI maintenant - t_activite ≥ CONFIG_TIMEOUT_MS ALORS
                Afficher("30 min sans activite")
                Changer_Mode(MODE_STANDARD)
            FIN SI
            RETOURNER                       // pas d'acquisition en configuration

        CAS MODE_STANDARD :
            SI long_rouge ALORS Changer_Mode(MODE_MAINTENANCE)
            SINON SI long_vert ALORS Changer_Mode(MODE_ECONOMIQUE)
            FIN SI

        CAS MODE_ECONOMIQUE :
            SI long_rouge ALORS Changer_Mode(MODE_MAINTENANCE) FIN SI

        CAS MODE_MAINTENANCE :
            SI long_rouge ALORS Changer_Mode(mode_precedent) FIN SI
    FIN SELON

    // Une mesure est en cours : on fait avancer l'acquisition
    SI Capteurs_EnCours() ALORS
        SI Capteurs_Tache() ALORS           // VRAI quand tout est lu ou TIMEOUT écoulé
            Fin_Mesure()
        FIN SI
    // Sinon : est-ce le moment de lancer une nouvelle mesure ?
    SINON SI mesure_immediate OU (maintenant - t_mesure ≥ Periode()) ALORS
        mesure_immediate ← FAUX
        t_mesure ← maintenant
        Mesure()
    FIN SI
FIN
```

7. Algorithmes des modes et des boutons

7.1 Détection d'un appui long avec anti-rebond

```
FONCTION Appui_Long(b : Bouton, maintenant : entier) : booléen
VARIABLES lecture : booléen
DEBUT
    lecture ← (Lire_Broche(b.port, b.broche) = 0)     // pull-up : appuyé = 0

    SI lecture ≠ b.brut ALORS                          // le niveau vient de changer
        b.brut ← lecture
        b.t_change ← maintenant
    FIN SI

    // l'état n'est validé que s'il est stable depuis 30 ms
    SI b.enfonce ≠ b.brut ET maintenant - b.t_change ≥ ANTI_REBOND_MS ALORS
        b.enfonce ← b.brut
        SI b.enfonce ALORS
            b.t_appui ← maintenant
            b.long_fait ← FAUX
        FIN SI
    FIN SI

    SI b.enfonce ET NON b.long_fait ET maintenant - b.t_appui ≥ APPUI_LONG_MS ALORS
        b.long_fait ← VRAI
        RETOURNER VRAI                                 // signalé une seule fois
    FIN SI
    RETOURNER FAUX
FIN
```

7.2 Changement de mode

```
PROCEDURE Changer_Mode(nouveau : ModeSysteme)
DEBUT
    SI mode = MODE_MAINTENANCE ET nouveau ≠ MODE_MAINTENANCE ALORS
        Logger_Monter()                      // on remet la carte SD en service
        Afficher("Carte SD remontee, enregistrement repris")
    FIN SI
    SI nouveau = MODE_MAINTENANCE ALORS
        mode_precedent ← mode                // pour y revenir à la sortie
        Logger_Demonter()                    // la carte peut être retirée
    FIN SI
    Capteurs_Annuler()                       // une mesure en cours est abandonnée

    mode ← nouveau

    SELON nouveau
        CAS MODE_STANDARD      : LED_Couleur(VERTE)
        CAS MODE_CONFIGURATION : LED_Couleur(JAUNE)
        CAS MODE_ECONOMIQUE    : LED_Couleur(BLEUE)
        CAS MODE_MAINTENANCE   : LED_Couleur(ORANGE)
    FIN SELON

    Afficher("=== Mode ", NOM[nouveau], " ===")
    SI nouveau = MODE_CONFIGURATION ALORS
        t_activite ← HAL_GetTick()
        Afficher("Acquisition desactivee. Tapez HELP.")
    FIN SI

    t_mesure ← HAL_GetTick()
    mesure_immediate ← VRAI                  // première mesure dès l'entrée dans le mode
FIN
```

7.3 Période entre deux mesures

```
FONCTION Periode() : entier       // en millisecondes
VARIABLES p : entier
DEBUT
    p ← cfg.log_interval × 60 000            // LOG_INTERVAL est en minutes
    SI mode = MODE_ECONOMIQUE ALORS
        p ← p × 2
    FIN SI
    SI mode = MODE_MAINTENANCE ALORS
        p ← MAINTENANCE_PERIODE_MS           // 5 s
    FIN SI
    RETOURNER p
FIN
```

7.4 Réalisation d'une mesure

Une mesure se fait en deux temps pour ne pas bloquer la boucle : `Mesure()` prend l'heure et lance l'acquisition, puis `Fin_Mesure()` écrit la ligne quand tous les capteurs ont répondu.

```
PROCEDURE Mesure()
VARIABLES avec_gps : booléen
DEBUT
    num_mesure ← num_mesure + 1
    avec_gps ← VRAI
    SI mode = MODE_ECONOMIQUE ALORS
        avec_gps ← (num_mesure MOD 2 = 0)            // GPS une mesure sur deux
    FIN SI

    // 1. Horodatage (gardé pour la ligne finale)
    SI NON RTC_Lire(t_horodatage, d_horodatage) ALORS
        erreur_rtc ← VRAI
        Erreurs_LED()
        Afficher("Erreur d'acces a l'horloge RTC")
        RETOURNER
    FIN SI
    erreur_rtc ← FAUX

    // 2. Lancement de l'acquisition (suivie par Capteurs_Tache)
    Capteurs_Demarrer(avec_gps)
FIN
```

```
PROCEDURE Fin_Mesure()
VARIABLES ligne : chaîne
DEBUT
    // Ligne "JJ/MM/AAAA HH:MM:SS;T;H;P;L;GPS"
    ligne ← Formater_Date(d_horodatage, t_horodatage) + ";" + Capteurs_Formater() + "\r\n"

    SI mode = MODE_MAINTENANCE ALORS
        Afficher(ligne)                              // carte SD démontée : port série
    SINON
        etat_sd ← Logger_Ecrire(ligne, d_horodatage.annee, d_horodatage.mois,
d_horodatage.jour)
        Afficher("[SD] ", ligne)
    FIN SI

    Erreurs_LED()                                    // voir chapitre 11
FIN
```

8. Algorithmes d'acquisition des capteurs

L'acquisition est **découpée en petites étapes** : `Capteurs_Demarrer()` prépare la mesure, puis `Capteurs_Tache()`, appelée à chaque tour de la boucle principale, essaie de lire les capteurs qui n'ont pas encore répondu. Elle s'arrête quand tout est lu ou quand `TIMEOUT` est écoulé. Pendant ce temps, les boutons et la LED continuent de fonctionner. Deux timeouts consécutifs d'un même capteur déclenchent une erreur.

8.1 Acquisition globale

```
PROCEDURE Capteurs_Demarrer(avec_gps : booléen)
DEBUT
    Effacer(mesures)
    mesures.gps_lu ← avec_gps
    // un capteur désactivé dans la configuration est considéré comme déjà lu
    dht_fini ← NON (cfg.temp_air = 1 OU cfg.hygr = 1)
    lum_fini ← NON (cfg.lumin = 1)
    gps_fini ← NON avec_gps
    t_debut ← HAL_GetTick()
    gps_data_ready ← FAUX                          // on attend une trame récente
    en_cours ← VRAI
FIN
```

```
FONCTION Capteurs_Tache() : booléen                // VRAI quand l'acquisition est finie
VARIABLES maintenant, t, h, lux : entier
DEBUT
    SI NON en_cours ALORS RETOURNER VRAI FIN SI
    maintenant ← HAL_GetTick()

    // --- DHT11 : une tentative toutes les 2 s jusqu'à réussite ---
    SI NON dht_fini ET maintenant - t_dht ≥ 2000 ALORS
        t_dht ← maintenant
        SI DHT11_Lire(t, h) ALORS
            mesures.temp ← t ; mesures.temp_ok ← (cfg.temp_air = 1)
            // l'hygrométrie n'est enregistrée que si HYGR_MINT ≤ t ≤ HYGR_MAXT
            mesures.hygr ← h
            mesures.hygr_ok ← (cfg.hygr = 1) ET (t ≥ cfg.hygr_mint) ET (t ≤ cfg.hygr_maxt)
            dht_fini ← VRAI
        FIN SI
    FIN SI

    // --- BH1750 : une tentative toutes les 200 ms ---
    SI NON lum_fini ET maintenant - t_lum ≥ 200 ALORS
        t_lum ← maintenant
        SI BH1750_Lire(lux) ALORS
            mesures.lux ← lux ; mesures.lum_ok ← VRAI ; lum_fini ← VRAI
        FIN SI
    FIN SI

    // --- Pression : pas de capteur câblé ---
    mesures.pres_ok ← FAUX                         // CAPTEUR_PRESSION_PRESENT = 0 -> "NA"

    // --- GPS : position extraite de la dernière trame reçue ---
    SI NON gps_fini ET GPS_Lire_Position() ALORS
        mesures.gps_ok ← VRAI ; gps_fini ← VRAI
    FIN SI

    // --- Fin : tout est lu, ou TIMEOUT écoulé ---
    SI (dht_fini ET lum_fini ET gps_fini) OU (maintenant - t_debut ≥ cfg.timeout × 1000)
ALORS
        Fin_Acquisition()
        RETOURNER VRAI
    FIN SI
    RETOURNER FAUX
FIN
```

```
PROCEDURE Fin_Acquisition()
DEBUT
    // compteur de timeouts consécutifs : remis à 0 si le capteur a répondu
    SI capteur DHT11 actif ALORS
        SI dht_fini ALORS nb_timeout[DHT11] ← 0 SINON nb_timeout[DHT11] ← nb_timeout[DHT11]
+ 1 FIN SI
    FIN SI
    SI cfg.lumin = 1 ALORS
        SI lum_fini ALORS nb_timeout[BH1750] ← 0 SINON nb_timeout[BH1750] ←
nb_timeout[BH1750] + 1 FIN SI
    FIN SI
    SI mesures.gps_lu ALORS
        SI mesures.gps_ok ALORS nb_timeout[GPS] ← 0 SINON nb_timeout[GPS] ← nb_timeout[GPS]
+ 1 FIN SI
    FIN SI

    erreurs.capteur    ← (nb_timeout[DHT11] ≥ 2) OU (nb_timeout[BH1750] ≥ 2)
    erreurs.gps        ← (nb_timeout[GPS] ≥ 2)
    erreurs.incoherent ← Valeurs_Incoherentes()
    en_cours ← FAUX
FIN
```

```
FONCTION Valeurs_Incoherentes() : booléen
DEBUT
    SI mesures.temp_ok ET (mesures.temp < cfg.min_temp_air OU mesures.temp >
cfg.max_temp_air) ALORS
        RETOURNER VRAI
    FIN SI
    SI mesures.hygr_ok ET (mesures.hygr < 0 OU mesures.hygr > 100) ALORS
        RETOURNER VRAI
    FIN SI
    SI mesures.pres_ok ET (mesures.pression < cfg.pressure_min OU mesures.pression >
cfg.pressure_max) ALORS
        RETOURNER VRAI
    FIN SI
    RETOURNER FAUX
FIN
```

```
FONCTION Capteurs_Formater() : chaîne        // "T;H;P;L;GPS", "NA" si absent
VARIABLES s : chaîne
DEBUT
    SI mesures.temp_ok ALORS s ← EnTexte(mesures.temp) SINON s ← "NA" FIN SI
    s ← s + ";"
    SI mesures.hygr_ok ALORS s ← s + EnTexte(mesures.hygr) SINON s ← s + "NA" FIN SI
    s ← s + ";"
    SI mesures.pres_ok ALORS s ← s + EnTexte(mesures.pression) SINON s ← s + "NA" FIN SI
    s ← s + ";"
    SI mesures.lum_ok ALORS s ← s + EnTexte(mesures.lux) SINON s ← s + "NA" FIN SI
    s ← s + ";"
    SI mesures.gps_ok ALORS s ← s + mesures.gps SINON s ← s + "NA" FIN SI
    RETOURNER s
FIN
```

8.2 DHT11 (bus 1 fil sur PB0)

```
FONCTION DHT11_Lire(SORTIE temperature, SORTIE humidite : entier) : booléen
VARIABLES octets : tableau[5] d'entiers ; i, j : entier
DEBUT
    // 1. Signal de départ : la carte tire la ligne à 0 pendant 18 ms
    Broche_En_Sortie(PB0)
    Ecrire_Broche(PB0, 0)
    Attendre(18 ms)
    Ecrire_Broche(PB0, 1)
    Attendre_us(30)
    Broche_En_Entree(PB0)

    // 2. Réponse du capteur : 80 µs à 0 puis 80 µs à 1 (sans interruption)
    Desactiver_Interruptions()
    SI NON Attendre_Niveau(PB0, 0, 1000 µs) ALORS
        Activer_Interruptions() ; RETOURNER FAUX          // capteur absent
    FIN SI
    SI NON Attendre_Niveau(PB0, 1, 1000 µs) OU NON Attendre_Niveau(PB0, 0, 1000 µs) ALORS
        Activer_Interruptions() ; RETOURNER FAUX
    FIN SI

    // 3. Lecture des 40 bits : 5 octets
    POUR i DE 0 À 4 FAIRE
        octets[i] ← 0
        POUR j DE 0 À 7 FAIRE
            Attendre_Niveau(PB0, 1, 1000 µs)              // début du bit
            Attendre_us(40)
            SI Lire_Broche(PB0) = 1 ALORS                 // impulsion longue = bit 1
                octets[i] ← octets[i] OU (1 décalé de (7 - j))
            FIN SI
            Attendre_Niveau(PB0, 0, 1000 µs)              // fin du bit
        FIN POUR
    FIN POUR
    Activer_Interruptions()

    // 4. Vérification de la somme de contrôle
    SI octets[4] = (octets[0] + octets[1] + octets[2] + octets[3]) MOD 256 ALORS
        humidite ← octets[0]
        temperature ← octets[2]
        RETOURNER VRAI
    FIN SI
    RETOURNER FAUX
FIN
```

8.3 BH1750 (I2C1, adresse 0x23)

```
PROCEDURE BH1750_Init()
DEBUT
    I2C_Envoyer(0x23, 0x10)                    // mode continu haute résolution (1 lux)
    Attendre(180 ms)                           // première mesure disponible
FIN

FONCTION BH1750_Lire(SORTIE lux : entier) : booléen
VARIABLES donnees : tableau[2] d'octets ; brut : entier
DEBUT
    SI I2C_Recevoir(0x23, donnees, 2, cfg.timeout) = OK ALORS
        brut ← donnees[0] × 256 + donnees[1]
        lux ← brut / 1,2
        RETOURNER VRAI
    SINON
        BH1750_Init()                           // relance si le capteur vient d'être
rebranché
        RETOURNER FAUX
    FIN SI
FIN
```

8.4 GPS (USART3, trames NMEA)

```
// Appelée par USART3_IRQHandler à chaque caractère reçu (stm32l4xx_it.c)
FONCTION Capteurs_GPS_IRQ() : booléen
VARIABLES c : caractère
DEBUT
    SI NON gps_actif ALORS RETOURNER FAUX FIN SI        // programme de test : la HAL s'en
charge
    SI erreur de réception (débordement…) ALORS Effacer_Erreur() FIN SI
    TANT QUE Caractere_Disponible(USART3) FAIRE
        c ← Lire_Caractere(USART3)
        SI c = '\n' OU c = '\r' ALORS
            SI gps_index > 0 ALORS
                gps_rx_buffer[gps_index] ← FIN_DE_CHAINE
                SI NON gps_data_ready ALORS              // la trame précédente a été lue
                    gps_trame ← gps_rx_buffer
                    gps_data_ready ← VRAI
                FIN SI
                gps_index ← 0
            FIN SI
        SINON SI gps_index < GPS_BUFFER_SIZE - 1 ALORS
            gps_rx_buffer[gps_index] ← c
            gps_index ← gps_index + 1
        SINON
            gps_index ← 0                                // trame trop longue : on
recommence
        FIN SI
    FIN TANT QUE
    RETOURNER VRAI
FIN

// Appelée par Capteurs_Tache : ne bloque pas
FONCTION GPS_Lire_Position() : booléen
VARIABLES champs : tableau de chaînes
DEBUT
    SI NON gps_data_ready ALORS RETOURNER FAUX FIN SI
    trame ← gps_trame ; gps_data_ready ← FAUX
    SI trame n'est pas "$GPGGA" ni "$GNGGA" ALORS RETOURNER FAUX FIN SI
    champs ← Decouper(trame, ',')
    SI champs[6] = "0" OU champs[2] est vide ALORS RETOURNER FAUX FIN SI   // pas de fix
    mesures.gps ← champs[2] + champs[3] + "," + champs[4] + champs[5]      //
"4851.1234N,00221.5678E"
    RETOURNER VRAI
FIN
```

8.5 Horloge RTC

```
FONCTION RTC_Lire(SORTIE heure : RTC_Heure, SORTIE date : RTC_Date) : booléen
DEBUT
    // l'heure doit toujours être lue avant la date (verrouillage du calendrier)
    SI HAL_RTC_GetTime(heure) ≠ OK ALORS RETOURNER FAUX FIN SI
    SI HAL_RTC_GetDate(date) ≠ OK ALORS RETOURNER FAUX FIN SI
    RETOURNER VRAI
FIN

PROCEDURE Init_RTC()
DEBUT
    Configurer_RTC(format 24 h, source LSE)
    SI Registre_Sauvegarde(DR0) = 0x32F2 ALORS
        RETOURNER                          // l'heure a déjà été réglée : on la garde
    FIN SI
    Regler_Heure(00:00:00) ; Regler_Date(01/01/2000)
FIN
```

9. Enregistrement sur carte SD

Les mesures d'un jour sont écrites dans le fichier `AAMMJJ_0.LOG` (exemple : `261016_0.LOG`). Quand ce fichier atteint `FILE_MAX_SIZE`, il est renommé en révision suivante (`_1`, `_2`…) et un nouveau fichier `_0` vide est créé. Les noms respectent le format 8.3 de FAT.

```
FONCTION Logger_Monter() : SdEtat
DEBUT
    SI monte ALORS RETOURNER SD_OK FIN SI
    SI f_mount(fs, "0:/") ≠ OK ALORS
        RETOURNER SD_ERREUR                    // carte absente ou non formatée en FAT32
    FIN SI
    monte ← VRAI
    RETOURNER SD_OK
FIN

PROCEDURE Logger_Demonter()
DEBUT
    f_mount(AUCUN, "0:/")
    monte ← FAUX
FIN
```

```
FONCTION Logger_Ecrire(ligne : chaîne, a, m, j : entier) : SdEtat
VARIABLES nom0 : chaîne ; fichier : FIL ; etat : SdEtat
DEBUT
    etat ← Logger_Monter()
    SI etat ≠ SD_OK ALORS RETOURNER etat FIN SI

    nom0 ← Format("%02d%02d%02d_0.LOG", a, m, j)
    SI f_open(fichier, nom0, OUVRIR_OU_CREER) ≠ OK ALORS RETOURNER Erreur() FIN SI

    // Le fichier dépasserait la taille maximale : archivage
    SI Taille(fichier) > 0 ET Taille(fichier) + Longueur(ligne) > cfg.file_max_size ALORS
        f_close(fichier)
        etat ← Archiver(nom0, a, m, j)
        SI etat ≠ SD_OK ALORS RETOURNER etat FIN SI
        f_open(fichier, nom0, CREER_NOUVEAU)
    FIN SI

    Aller_A_La_Fin(fichier)
    SI f_write(fichier, ligne) ≠ OK ALORS
        f_close(fichier) ; RETOURNER Erreur()
    FIN SI
    f_close(fichier)
    RETOURNER SD_OK
FIN
```

```
FONCTION Archiver(nom0 : chaîne, a, m, j : entier) : SdEtat
VARIABLES n : entier ; nom : chaîne
DEBUT
    POUR n DE 1 À 9 FAIRE
        nom ← Format("%02d%02d%02d_%d.LOG", a, m, j, n)
        SI NON Fichier_Existe(nom) ALORS
            f_rename(nom0, nom)
            RETOURNER SD_OK
        FIN SI
    FIN POUR
    RETOURNER SD_PLEINE                          // 9 archives déjà présentes pour ce jour
FIN

FONCTION Erreur(code) : SdEtat
DEBUT
    SI code = ACCES_REFUSE ALORS RETOURNER SD_PLEINE FIN SI   // plus de place
    Logger_Demonter()                                         // on remontera à la
prochaine écriture
    RETOURNER SD_ERREUR
FIN
```

Exemple de ligne enregistrée : `16/10/2026 20:15:00;21;48;NA;312;4851.1234N,00221.5678E`
Ordre des champs : date et heure ; température (°C) ; humidité (%) ; pression (hPa) ; luminosité (lux) ; position GPS.

10. Configuration et console série

En mode configuration, l'utilisateur tape des commandes dans un terminal série (115200 bauds). La saisie est lue caractère par caractère **sans bloquer** la boucle principale ; chaque commande validée par Entrée est exécutée puis les paramètres sont sauvegardés en flash.

Commande	Effet
`LOG_INTERVAL=10`, `TIMEOUT=30`…	Modifie un paramètre (valeur contrôlée entre min et max)
`RESET`	Remet tous les paramètres aux valeurs par défaut
`VERSION`	Affiche la version du programme et le numéro de lot
`PARAMS` / `HELP`	Affiche les paramètres et la liste des commandes
`CLOCK=HH:MM:SS`	Règle l'heure de la RTC
`DATE=MOIS,JOUR,ANNEE`	Règle la date de la RTC
`DAY=MON…SUN`	Règle le jour de la semaine

```
PROCEDURE Console(maintenant : entier)
VARIABLES c : caractère
DEBUT
    TANT QUE Caractere_Disponible(USART2) FAIRE
        c ← Lire_Caractere(USART2)
        t_activite ← maintenant
        SI c = '\r' OU c = '\n' ALORS
            SI lg > 0 ALORS
                ligne[lg] ← FIN_DE_CHAINE ; lg ← 0
                SI NON Config_Commande(ligne) ALORS
                    Afficher("Commande inconnue (HELP pour la liste)")
                FIN SI
                Afficher("> ")
            FIN SI
        SINON SI c = RETOUR_ARRIERE ET lg > 0 ALORS
            lg ← lg - 1
        SINON SI c est imprimable ET lg < TAILLE(ligne) - 1 ALORS
            ligne[lg] ← Majuscule(c) ; lg ← lg + 1
            Afficher(c)                                  // écho
        FIN SI
    FIN TANT QUE
FIN
```

```
FONCTION Config_Commande(cmd : chaîne) : booléen
VARIABLES nom, arg : chaîne ; v : entier ; i : entier
DEBUT
    Separer(cmd, '=', nom, arg)

    SI nom = "RESET"   ALORS Config_Defaut() ; Config_Sauver() ; RETOURNER VRAI FIN SI
    SI nom = "VERSION" ALORS Afficher(VERSION_PROGRAMME, NUMERO_LOT) ; RETOURNER VRAI FIN
SI
    SI nom = "PARAMS" OU nom = "HELP" ALORS Config_Afficher() ; RETOURNER VRAI FIN SI
    SI arg est vide ALORS RETOURNER FAUX FIN SI

    SI nom = "CLOCK" ALORS RETOURNER Cmd_Clock(arg) FIN SI
    SI nom = "DATE"  ALORS RETOURNER Cmd_Date(arg)  FIN SI
    SI nom = "DAY"   ALORS RETOURNER Cmd_Day(arg)   FIN SI

    // Recherche du paramètre dans la table PARAMS
    POUR i DE 0 À NB_PARAMS - 1 FAIRE
        SI nom = PARAMS[i].nom ALORS
            v ← EnEntier(arg)
            SI arg n'est pas un nombre OU v < PARAMS[i].min OU v > PARAMS[i].max ALORS
                Afficher("Valeur invalide pour ", nom)
            SINON
                PARAMS[i].valeur ← v
                Config_Sauver()
                Afficher(nom, " = ", v)
            FIN SI
            RETOURNER VRAI
        FIN SI
    FIN POUR
    RETOURNER FAUX
FIN

FONCTION Cmd_Clock(arg : chaîne) : booléen
VARIABLES h, m, s : entier
DEBUT
    Lire(arg, "HH:MM:SS", h, m, s)
    SI h ∉ [0..23] OU m ∉ [0..59] OU s ∉ [0..59] ALORS
        Afficher("Format : CLOCK=HH:MM:SS") ; RETOURNER VRAI
    FIN SI
    HAL_RTC_SetTime(h, m, s)
    Ecrire_Registre_Sauvegarde(DR0, 0x32F2)          // l'heure survit au reset
    RETOURNER VRAI
FIN
```

```
PROCEDURE Config_Charger()
DEBUT
    SI Flash[CFG_ADRESSE].magic = CFG_MAGIC ET Flash.somme = Somme(Flash.cfg) ALORS
        cfg ← Flash.cfg                              // paramètres sauvegardés valides
    SINON
        Config_Defaut()                              // première mise en route
    FIN SI
FIN

PROCEDURE Config_Sauver()
DEBUT
    image.magic ← CFG_MAGIC ; image.cfg ← cfg ; image.somme ← Somme(cfg)
    Deverrouiller_Flash()
    Effacer_Page(banque 2, page 255)
    POUR chaque mot de 64 bits de image FAIRE
        Programmer_Flash(CFG_ADRESSE + 8 × i, mot)
    FIN POUR
    Verrouiller_Flash()
FIN
```

11. LED d'état et gestion des erreurs

La LED affiche en temps normal la couleur du mode. Quand une erreur est active, elle clignote à 1 Hz entre le rouge et une seconde couleur qui identifie l'erreur.

Erreur	Clignotement (1 Hz)	Cause
ERR_RTC	Rouge / bleu, durées égales	Accès impossible à l'horloge RTC
ERR_GPS	Rouge / jaune, durées égales	Pas de position GPS 2 fois de suite
ERR_CAPTEUR	Rouge / vert, durées égales	Un capteur ne répond pas 2 fois de suite
ERR_INCOHERENT	Rouge / vert, vert 2× plus long	Valeur hors des seuils MIN/MAX
ERR_SD_PLEINE	Rouge / blanc, durées égales	Carte SD pleine
ERR_SD	Rouge / blanc, blanc 2× plus long	Carte absente ou erreur d'écriture

```
// Choisit le code d'erreur à afficher (le plus grave d'abord)
PROCEDURE Erreurs_LED()
DEBUT
    SI erreur_rtc ALORS LED_Erreur(ERR_RTC)
    SINON SI etat_sd = SD_ERREUR ALORS LED_Erreur(ERR_SD)
    SINON SI etat_sd = SD_PLEINE ALORS LED_Erreur(ERR_SD_PLEINE)
    SINON SI erreurs.capteur ALORS LED_Erreur(ERR_CAPTEUR)
    SINON SI erreurs.gps ALORS LED_Erreur(ERR_GPS)
    SINON SI erreurs.incoherent ALORS LED_Erreur(ERR_INCOHERENT)
    SINON LED_Erreur(AUCUNE)
    FIN SI
FIN
```

```
// Appelée toutes les 1 ms par l'interruption SysTick
PROCEDURE LED_Tick()
VARIABLES r, g, b, pas, t, duree_rouge : entier
DEBUT
    ms ← ms + 1
    SI NON actif ALORS Allumer(0, 0, 0) ; RETOURNER FIN SI

    r ← niv_r ; g ← niv_g ; b ← niv_b               // couleur du mode

    SI erreur ≠ AUCUNE ALORS
        t ← ms MOD 1000                              // position dans la seconde
        SI erreur = ERR_INCOHERENT OU erreur = ERR_SD ALORS
            duree_rouge ← 333                        // 2e couleur 2 fois plus longue
        SINON
            duree_rouge ← 500
        FIN SI
        SI t < duree_rouge ALORS
            r ← 10 ; g ← 0 ; b ← 0                   // rouge
        SINON
            SELON erreur
                CAS ERR_RTC                  : r, g, b ← 0, 0, 10    // bleu
                CAS ERR_GPS                  : r, g, b ← 10, 5, 0    // jaune
                CAS ERR_CAPTEUR, ERR_INCOHERENT : r, g, b ← 0, 10, 0 // vert
                AUTRE                        : r, g, b ← 10, 10, 10  // blanc
            FIN SELON
        FIN SI
    FIN SI

    // PWM logiciel : période 10 ms, 10 niveaux d'intensité
    pas ← ms MOD 10
    Allumer(pas < r, pas < g, pas < b)               // anode commune : allumé = broche à
0
FIN
```

12. Récapitulatif des fonctions par module

Module	Fonction	Rôle
main	`main()`	Initialisations puis boucle infinie
	`SystemClock_Config()`, `MX_xxx_Init()`	Horloges et périphériques (CubeMX)
modes	`Modes_Init()`	Mode de départ selon les boutons
	`Modes_Boucle()`	Transitions et cadence des mesures
	`Appui_Long(b, now) : booléen`	Anti-rebond et appui de 5 s
	`Changer_Mode(m)`	Couleur LED, carte SD, mode précédent
	`Periode() : entier`	Intervalle entre deux mesures
	`Mesure()`	Horodatage et lancement de l'acquisition
	`Fin_Mesure()`	Écriture de la ligne (SD ou port série)
	`Erreurs_LED()`	Choix du code d'erreur de la LED
	`Console(now)`	Saisie des commandes
capteurs	`Capteurs_Init()`	DWT (µs), BH1750, réception GPS
	`Capteurs_Demarrer(avec_gps)`	Prépare une acquisition
	`Capteurs_Tache() : booléen`	Fait avancer l'acquisition, VRAI à la fin
	`Capteurs_EnCours()`, `Capteurs_Annuler()`	État / abandon de l'acquisition
	`Fin_Acquisition()`	Compte les timeouts, met à jour les erreurs
	`DHT11_Lire(t, h) : booléen`	Température et humidité
	`BH1750_Init()`, `BH1750_Lire(lux) : booléen`	Luminosité
	`GPS_Lire_Position() : booléen`	Extraction de la position depuis $GPGGA
	`Capteurs_GPS_IRQ() : booléen`	Réception GPS sous interruption USART3
	`Valeurs_Incoherentes() : booléen`	Contrôle des seuils
	`Capteurs_Formater() : chaîne`	Ligne T;H;P;L;GPS
logger	`Logger_Monter() : SdEtat`	Montage FatFs
	`Logger_Demonter()`	Démontage (maintenance)
	`Logger_Ecrire(ligne, a, m, j) : SdEtat`	Ajout d'une ligne au fichier du jour
	`Archiver(nom0, a, m, j) : SdEtat`	Renommage en révision n
config	`Config_Charger()`	Lecture de la flash
	`Config_Sauver()`	Écriture en flash
	`Config_Defaut()`	Valeurs par défaut
	`Config_Afficher()`	Liste des paramètres
	`Config_Commande(cmd) : booléen`	Interprétation d'une commande
	`Cmd_Clock()`, `Cmd_Date()`, `Cmd_Day()`	Réglage de la RTC
	`Somme(c) : entier`	Somme de contrôle des paramètres

led	`LED_Activer()`	Démarre l'affichage
	`LED_Couleur(r, g, b)`	Couleur du mode (0..10)
	`LED_Erreur(e)`	Code d'erreur à afficher
	`LED_Tick()`	PWM et clignotement (SysTick 1 ms)
tests	`Test_LED()`, `Test_Bouton()`, `Test_Serial()`, `Test_DHT11()`, `Test_RTC()`, `Test_BH1750()`, `Test_SD()`, `Test_GPS()`	Validation de chaque composant séparément

13. Conclusion et suite du projet

Cette architecture sépare clairement le **matériel** (pilotes HAL), les **composants** (capteurs, carte SD, LED) et la **logique** de la station (modes et configuration). Tous les composants ont été validés individuellement grâce aux programmes de test, puis intégrés : les 4 modes, l'acquisition des capteurs, l'enregistrement sur carte SD, la LED et la console de configuration sont codés et le projet compile.

Les prochaines étapes sont :

- tester la station complète avec tous les composants branchés en même temps ;
- activer les codes d'erreur clignotants de la LED une fois tous les composants câblés ;
- **optimiser** les algorithmes : lecture du DHT11 sans attente active, mise en veille du microcontrôleur entre deux mesures en mode économique, réduction des temps d'attente actifs.